Git, GitHub and R with the kill-curve code

Oct 29, 2026 · Canan

What this session covers

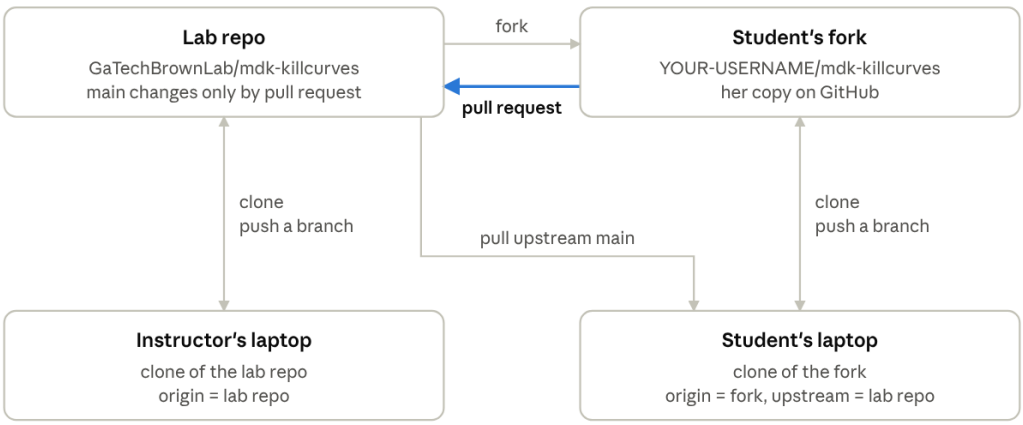
By the end, the student has her own fork and clone of the lab's kill-curve repo, has opened a pull request to it, and can read and change the R script that turns plate counts into MDK99 and MDK99.99.

Two ideas carry the Git part:

- A **remote** is a named address of another copy of the repo. `origin` is the copy you cloned from. `upstream` is the lab repo, when `origin` is your fork.
- The lab repo's `main` branch changes only through **pull requests**. Nobody pushes to it directly, the instructor included.

The repo used throughout is `GaTechBrownLab/mdk-killcurves`. Replace `YOUR-USERNAME` with the student's GitHub username wherever it appears.

Changes reach the lab repo only through pull requests



remotes and pull requests · 2 GitHub repos, 2 laptops

The student never pushes to the lab repo: she pushes branches to her fork and opens pull requests from there. The lab's changes come back to her laptop from `upstream`.

Before the session

The session runs smoothly only if sign-in works beforehand; most lost time in a first Git lesson is authentication.

Student

- ☐ GitHub account with two-factor authentication turned on.
- ☐ Invitation to the GaTechBrownLab organization accepted (the instructor sends it from the organization's People page).
- ☐ Git installed. In Terminal, `git --version` prints a version. On a Mac without it, `xcode-select --install` installs it.
- ☐ Name and email set, using the email on the GitHub account (commands below).
- ☐ Terminal sign-in done: install the GitHub CLI (`brew install gh`), run `gh auth login`, and choose GitHub.com, HTTPS, "Authenticate Git with your GitHub credentials", then "Login with a web browser".
- ☐ R and RStudio installed, with the ggplot2 package.
- ☐ The plate-count sheet `(complete)_20261019_1058_evolvedT1-3_MDK_TOB_completed.csv` from the lab Dropbox. It is not in the repo.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

The last line tells `git pull` to merge when your branch and the lab repo have both moved on. Without it, Git stops with "Need to specify how to reconcile divergent branches". The instructor needs it too.

Part 1: Instructor demo, putting the code in the lab repo

The instructor's own code goes in through a pull request too, so the student sees the whole cycle once before doing it.

1. On GitHub, open GaTechBrownLab and choose **New repository**. Name it `mdk-killcurves`, pick the visibility, and tick only **Add a README file**. The license and `.gitignore` come with the code; the stub README gives `main` a first commit for the pull request to compare against.
2. Clone it and look at the remotes. Here `origin` is the lab repo itself, and there is no `upstream`, because this clone came straight from the lab.

```
cd ~/Documents/GitHub
git clone https://github.com/GaTechBrownLab/mdk-killcurves.git
cd mdk-killcurves
git remote -v
```

3. Make a branch for the work. `git branch` marks the current branch with `*`.

```
git switch -c add-killcurve-code
git branch
```

4. Copy the prepared files in, then the data sheet into `data/`. `git status` lists the scripts, `LICENSE`, `.gitignore` and `data/README.md`, and shows `README.md` as modified, but it does not list the CSV. That is `.gitignore` at work.

```
git status
```

5. Stage and commit.

```
git add -A
git commit -m "Add MDK kill-curve analysis"
```

6. Push the branch. `-u` links the local branch to the one on GitHub, so later pushes are just `git push`.

```
git push -u origin add-killcurve-code
```

7. On the repo page, click **Compare & pull request**. Check that it reads base `main` ← compare `add-killcurve-code`, write what the code does, and create the pull request. Show the **Files changed** tab: this is where review happens.
8. Click **Merge pull request**, confirm, then **Delete branch**. Bring the merge back to the laptop and remove the local branch:

```
git switch main
git pull
git branch -d add-killcurve-code
```

Part 2: Student, your own copy

You work in your fork and send changes to the lab repo by pull request; your laptop knows both addresses.

1. On the lab repo page, click **Fork**, choose your account as the owner, and create the fork. You now have `YOUR-USERNAME/mdk-killcurves` on GitHub.
2. Clone your fork, not the lab repo:

```
cd ~/Documents/GitHub
git clone https://github.com/YOUR-USERNAME/mdk-killcurves.git
cd mdk-killcurves
```

3. Add the lab repo as `upstream`. You do this once per clone.

```
git remote add upstream https://github.com/GaTechBrownLab/mdk-killcurves.git
git remote -v
```

You should see four lines: `origin` pointing to your fork and `upstream` pointing to GaTechBrownLab, each once for fetch and once for push.

4. Copy the plate-count CSV into `data/` and run `git status`. It says there is nothing to commit. Why does the lab keep raw data out of Git, and where does it live instead?
5. In RStudio, choose **File → New Project → Existing Directory** and pick this folder. RStudio writes `mdk-killcurves.Rproj`, and `git status` now lists it as untracked. Leave it for Exercise 2.

Part 3: Student, your first pull request

The first change is a real one: line 10 of `mdk_killcurves.R` says "Base R only. If ggplot2 is installed it is used for the figures.", but line 273 loads ggplot2 unconditionally, so the script stops without it.

1. Start from an up-to-date `main`, then branch:

```
git switch main
git pull upstream main
git switch -c fix-header-comment
```

2. Change line 10 to say the figures need ggplot2, keeping it on one line so the line numbers in this worksheet still match, and save.
3. Look at the change, then stage and commit it:

```
git diff
git add mdk_killcurves.R
git commit -m "Header comment: figures need ggplot2"
```

4. Push the branch to your fork. `origin` is your fork, so nothing reaches the lab repo yet.

```
git push -u origin fix-header-comment
```

5. On your fork's page, click **Compare & pull request**. Check both sides before creating it: base repository `GaTechBrownLab/mdk-killcurves`, base `main` ← head repository `YOUR-USERNAME/mdk-killcurves`, compare `fix-header-comment`. Say what you changed and why.
6. The instructor reviews in **Files changed** and may ask for a change. Commit and `git push` again on the same branch; the pull request updates itself.
7. The instructor merges. Your laptop and your fork do not have the merge yet; Part 4 brings it back.

Part 4: Keeping your fork in step with the lab repo

A merged pull request changes only the lab repo; you bring it to your laptop from `upstream`, then to your fork through `origin`. Do this at the start of every work session and before every new branch.

```
git switch main
git pull upstream main
git push origin main
git branch -d fix-header-comment
```

The fork page's **Sync fork** button does the same on GitHub; after it, a plain `git pull` updates the laptop.

`git branch -d` deletes a branch only once its work is in `main`. If it refuses, `main` is not up to date yet, or the pull request was squashed when merged; check before using `-D`, which deletes regardless.

When two people change the same lines, `git pull` stops with a merge conflict and marks the file like this:

```
<<<<<<< HEAD
    title    = "your version of the line",
=======
    title    = "the lab repo's version of the line",
>>>>>>> a08a301ccde028de3009bf010e36889eb9ae786d
```

`HEAD` is your branch; the long id is the lab repo's commit. Edit the file to the version you want, delete the three marker lines, then `git add` the file and `git commit`. Exercise 4 sets one up on purpose.

Kill curves in brief

A kill curve follows how many cells are still alive after an antibiotic is added. Here it measures how long tobramycin takes to kill 99% and 99.99% of each evolved line: the MDK99 and MDK99.99, the minimum duration for killing (Balaban et al. 2019, *Nat Rev Microbiol* 17:441-448).

The experiment

Part	In this data set
Lines	Three evolved lines of *Neisseria subflava*, T1-T3

Part	In this data set
Doses	Tobramycin at 0 (no-drug control), 0.4, 2, 4, 24 and 100 µg/ml; all doses of a line start from the same culture
Samples	0, 1, 2, 4, 6, 8, 24 and 48 h after the drug is added
Counting	Each sample is diluted in tenfold steps (neat to 10^{-7}) and spotted on plates, mostly 5 or 10 µl per spot and four spots per dilution; colonies are counted after incubation

From colonies to an MDK

1. **Density** (script section 2). Colony-forming units (CFU) per ml = mean colonies per spot ÷ spot volume in ml ÷ dilution. For example, 25 colonies in a 10 µl spot of the 10^{-4} dilution is 25 ÷ 0.01 ÷ 10^{-4} = 2.5 × 10^{7} CFU/ml.
2. **One dilution per time point** (section 3). Only spots with 3-100 colonies are trusted: fewer is mostly chance, and with more the colonies merge. The script keeps the dilution closest to the middle of that window.
3. **Survival** (section 4). Each density is divided by the line's starting density N_0 and put on a log scale. N_0 is the median density at 0 h across doses 0, 0.4, 2 and 4, because all doses of a line come from one culture.

```
\text{survival}(t) = \log_{10} \frac{N(t)}{N_0}
```

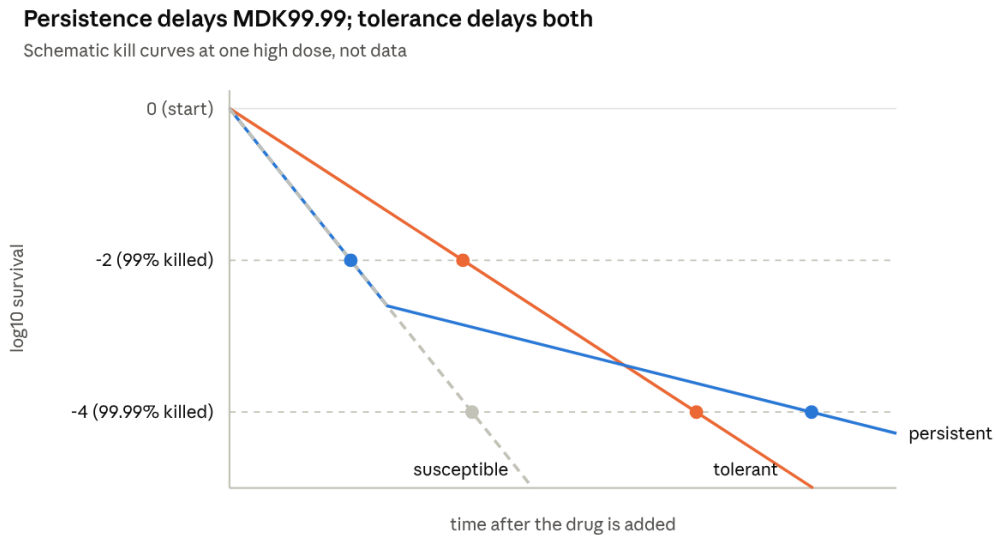
So 0 means nothing has died, -1 means 90% killed, -2 means 99% killed and -4 means 99.99% killed.

4. **MDK** (section 5). MDK99 is the first time survival reaches -2, and MDK99.99 the first time it reaches -4. Between two samples the script draws a straight line: if survival is -1 at 4 h and -3 at 8 h, it reaches -2 halfway, at 6 h.

Below detection. No colonies on a plate does not mean no cells. The script places such points at the smallest density the plates could detect, one colony in all the volume examined, and the figure draws them as open triangles. An MDK reached through one of them is written `<= x`, at or before x hours; `> 48 h` means the curve never got there.

Reading the shape (Balaban et al. 2019)

- **Resistance:** the cells grow at the dose, so the curve does not fall. It is measured by the MIC, not by kill curves.
- **Tolerance:** the whole population dies more slowly. MDK99 is longer and the MIC is unchanged.
- **Persistence:** most cells die at the normal rate, but a small subpopulation dies much more slowly. The curve bends (biphasic): MDK99 is normal and MDK99.99 is disproportionately long.



schematic kill curves · susceptible, tolerant, persistent

Each dot is an MDK: where a curve first crosses the 99% or the 99.99% line. The persistent population dies like the susceptible one at first and shares its MDK99, but its few survivors push MDK99.99 latest of all; the tolerant one is slow at both thresholds.

Questions

1. Why does the experiment include a no-drug control, and what would it mean if the control fell below -2?
2. To call a line tolerant or persistent, what do you need to compare its MDKs with?
3. Why is N_0 the median of several doses at 0 h, rather than each dose's own 0 h count? The comment at lines 196-199 of the script answers it.

Part 5: R basics with the kill-curve script

The script turns one plate-count sheet into one table of MDK99 and MDK99.99 per lineage and dose; you run it in pieces and ask what each piece did. Open `mdk_killcurves.R` in the RStudio project from Part 2. To run some lines, select them and press Cmd+Enter; **Source** runs the whole file.

1. Where R is looking. Run `getwd()` and `list.files("data")`. The project sets the working directory to the repo folder, so `"data/..."` works on any computer. That is why the script has no `setwd()`.

2. Reading the sheet. Run line 15, then:

```
dim(d)
names(d)
head(d)
str(d)
```

Check: 432 rows and 12 columns. Which columns hold numbers, and which hold text that should be numbers?

3. Types and missing values. Try this in the Console:

```
as.numeric(c("12", "", "TNTC"))
```
It returns `12 NA NA` with a warning. Read the comments at lines 45-60: why is a missing spot (`NA`) not the same as a zero, and when does the script treat a blank plate as a real zero?

4. Selecting rows and columns. Run lines 66-80, which fill the merged cells down and make the columns numeric. Then:

```
table(d$Lineage, d$Treatment)
d[d$t == 48 & d$Treatment == 100, c("Sample ID", "Dilution", "Rep1", "Notes")]
```
The first gives 24 rows for every lineage and dose. The second gives nine rows with no counts, three of them noted "No colony counts recorded". Why does `Sample ID` need quotes here, and backticks with `$` (`d$`Sample ID``)?

5. Functions. Run `fill_down(c("0h", "", "", "4h", ""))` and explain the result. Then run lines 215-229 to define `mdk()`. Predict the answer before running each line:

```
mdk(time = c(0, 4, 8, 24), logS = c(0, -1, -3, -5), cens = rep(FALSE, 4), target = -2)
mdk(time = c(0, 4, 8, 24), logS = c(0, -1, -3, -5), cens = rep(FALSE, 4), target = -4)
mdk(time = c(0, 4, 8, 24), logS = c(0, -0.5, -1, -1.5), cens = rep(FALSE, 4), target = -2)
```
6. Settings. Lines 37-64 hold every analysis decision as a named value, such as `COUNT_MIN <- 3`, the fewest colonies a spot needs to count. Why put these at the top instead of inside the code?

7. The whole script. Click **Source**. It prints the zeros it treated as real, the inoculum and the MDK table, and writes `mdk_points.csv`, `mdk_summary.csv` and a PNG. Run `git status`: none of them appear. Find the lines in `.gitignore` that keep them out.

8. The figure, layer by layer. Open `mdk_plot_ggplot.R` and run it. Then change one thing at a time and rerun: put `#` before `geom_line(linewidth = 0.7) +`, change a colour in `dose_cols`, delete `facet_wrap(~ Lineage) +`. Undo each change with `git restore mdk_plot_ggplot.R`.

Instructor key for step 5: 6 h, 16 h, and `NA`, because the curve never reaches 99% killing.

Exercises

Each exercise starts with a question about the three lines and ends with an answer you can check in the key at the end. Start each one from an up-to-date `main` (Part 4), and Source `mdk_killcurves.R` first so that `out`, the MDK table, and `pts`, the points, are in your Environment.

1. Which line survives the highest dose longest? No file changes, so no branch is needed. Run each line and read the result:

```
out[out$Dose == 100, ]
out[out$Lineage == "T2", ]
out[!grepl(">", out$MDK9999), c("Lineage", "Dose", "MDK9999")]
```
The first keeps the rows where the dose is 100. The third keeps the rows whose MDK99.99 does not contain ">" (`!` means not), that is, the curves that reached 99.99% killing within 48 h. At 100 µg/ml, which line reaches 99% killing first, and which takes longest to reach 99.99%? Which doses reach 99.99% killing at all? Check each answer against the figure.

2. Put the lines side by side. The figure has one panel per line, so comparing the lines at one dose means looking across three panels. Make a second figure with one panel per dose and one colour per line:

```
git switch -c plot-by-dose
cp mdk_plot_ggplot.R mdk_plot_by_dose.R
```

In `mdk_plot_by_dose.R`:

- Line 42: change both `Dose` to `Lineage`.
- Line 56: change `facet_wrap(~ Lineage)` to `facet_wrap(~ Dose)`.
- Lines 58-59: replace them with the two lines below.
- Line 75: change the legend title to `"Lineage"`.
- Line 97: change the file name to `"mdk_killcurves_by_dose.png"` and `height = 3.8` to `height = 6`.

```
  scale_colour_manual(values = c(T1 = "#1b9e77", T2 = "#d95f02",
                                 T3 = "#7570b3")) +
```

Run it. At 24 µg/ml, which line dies most slowly? Is it the same line at 100 µg/ml? The first figure shades the doses from light to dark; why do the lines get three unrelated colours instead? Lines 19-23 of the script explain the dose colours. Then commit only the new file, push, and open a pull request:

```
git add mdk_plot_by_dose.R
git commit -m "Kill curves with one panel per dose"
git push -u origin plot-by-dose
```

3. Does dropping the 4 h samples change the answer? The 4 h dilutions were wrong for samples S01-S12, so line 41 drops those. Line 42 would drop every 4 h sample. A branch you never merge is a safe place to try it:

```
git switch -c try-settings
```

In the Console, keep the current table with `before <- out`. Change `FALSE` to `TRUE` on line 42, Source again, and put the two tables side by side:

```
cbind(before[c("Lineage", "Dose", "MDK99", "MDK9999")], new_MDK99 = out$MDK99,
new_MDK9999 = out$MDK9999)
```

Which numbers changed, by how much, and does either change which line is fastest at that dose? Then throw the experiment away. Uncommitted changes follow you to another branch, so restore the file first:

```
git restore mdk_killcurves.R
git switch main
git branch -d try-settings
```

4. Add MDK90, the time to kill 90%. `mdk()` takes the threshold as its `target`, so a new threshold needs one more call, not a new function.

```
git switch -c add-mdk90
```

Add these three lines, each below the line named in its comment. Start at the bottom, so the line numbers above stay right:

```
# below line 256, in the table that is printed (out)
                  MDK90   = fmt(res$MDK90,   res$MDK90_cens,   res$t_max),
# below line 242, in the numeric table (res)
    MDK90     = round(m1$t, 2), MDK90_cens   = m1$cens,
# below line 234
  m1 <- mdk(g$t, g$log_surv, g$censored, -1)
```

Source it and read the new MDK90 column. One answer looks wrong: T3's no-drug control reaches 90% killing at 0.84 h. Look at its first three samples:

```
pts[pts$Lineage == "T3" & pts$Treatment == 0 & pts$t <= 2, c("t", "Dilution",
"mean_ct", "log_surv")]
```

Did 90% of the cells really die and then grow back within an hour? What does this say about -1 as a threshold? Commit, push and open the pull request with your answer in the description. The instructor decides whether to merge it; closing a pull request without merging is a normal ending too.

Command reference

Task	Command
Check Git is installed	`git --version`
Set name and email	`git config --global user.name "Your Name"` and `git config --global user.email "you@example.com"`
New repos start on main	`git config --global init.defaultBranch main`
Pull merges when both sides moved on	`git config --global pull.rebase false`
Sign in from the terminal	`gh auth login`
Clone your fork	`git clone https://github.com/YOUR-USERNAME/REPO.git`
Add the lab repo as upstream	`git remote add upstream https://github.com/GaTechBrownLab/REPO.git`
List remotes	`git remote -v`
See what changed	`git status`, then `git diff`
Update from the lab	`git switch main`, `git pull upstream main`, `git push origin main`
Start a branch	`git switch -c BRANCH`
Move to a branch	`git switch BRANCH`
Stage	`git add FILE` or `git add -A`
Commit	`git commit -m "What and why"`
First push of a branch	`git push -u origin BRANCH`

Task	Command
History	`git log --oneline --graph`
Discard uncommitted changes to a file	`git restore FILE`
Delete a merged branch	`git branch -d BRANCH`